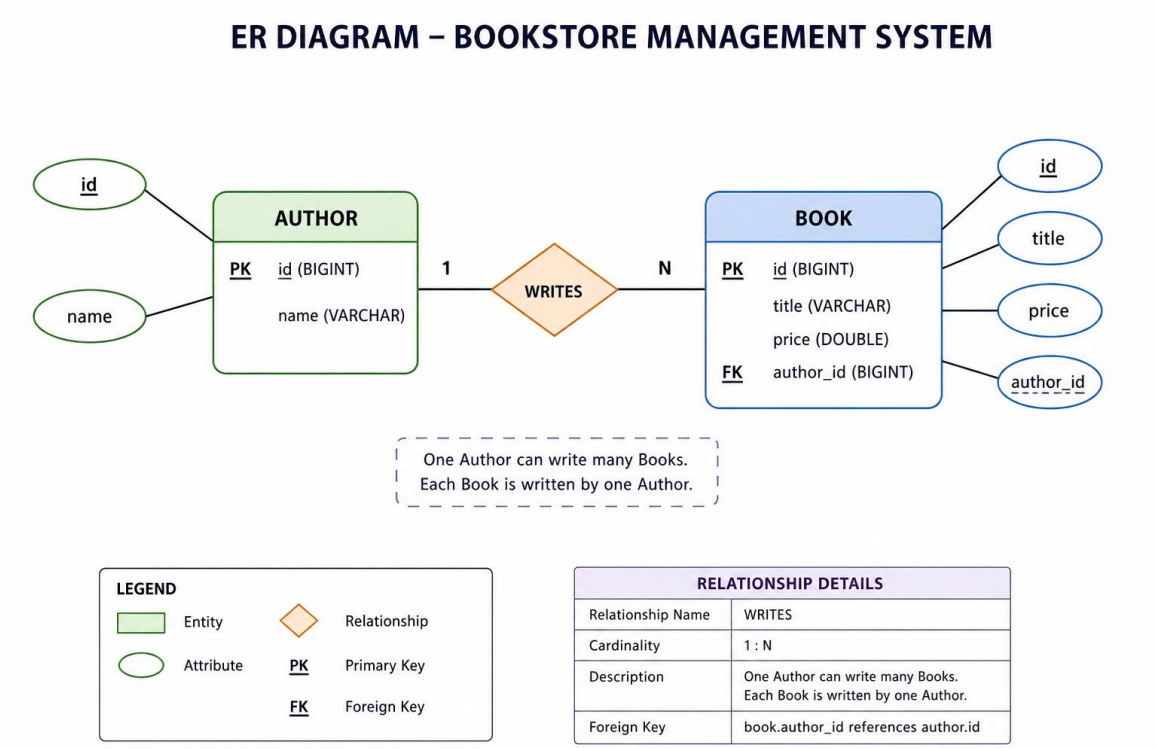
Project Title: Spring Boot Bookstore CRUD Application



BOOKSTORE PROJECT LINK

GITHUB LINK --->
https://github.com/Sanskriti10247/spring-boot-bookstore-crud.git

○ Overview

This project is a Spring Boot web application using JSP, MySQL, and JPA. It manages Books and Authors with create, read, and update operations.

– Entities

We used two entities:

- **Author**
- **Book**

Relationship:

- One Author → Many Books

- Many Books → One Author

Annotations used:

- @Entity
- @OneToMany
- @ManyToOne

★ Architecture

- **Tables:**
- author (id, name)
- book (id, title, price, author_id)

Controller -> Service -> Repository -> Database

★ Features

- ♦ Create books
- ♦ view books
- ♦ update books
- ♦ join query using JPA.

-> Database Design

Two tables: **author and book**. Foreign key relationship using *author_id.*

-> Custom Query

SELECT b FROM Book b JOIN b.author a

- ## CREATE OPERATION

- JSP form: add-book.jsp
- Controller: /saveBook
- Data saved using JPA repository

- ## READ OPERATION

- JSP: list-books.jsp
- Controller fetches data from service

- Custom query:
- @Query("SELECT b FROM Book b JOIN b.author a")

• UPDATE OPERATION

- Edit page: edit-book.jsp
- Controller: /updateBook
- Updates existing records

• EXCEPTION HANDLING

- Used:

- @ExceptionHandler(Exception.class)

- To prevent application crashes and show user-friendly errors.

• SERVICE LAYER

- Business logic handled using:

- BookService
- AuthorService

• REPOSITORY LAYER

- JpaRepository used
- Custom JOIN query implemented

• VIEW LAYER

- JSP pages:

- Add Book
- List Books
- Edit Book

- Styled using CSS for better UI.

- ## TESTING

- Unit tests implemented using:

- JUnit
- Spring Boot Testing

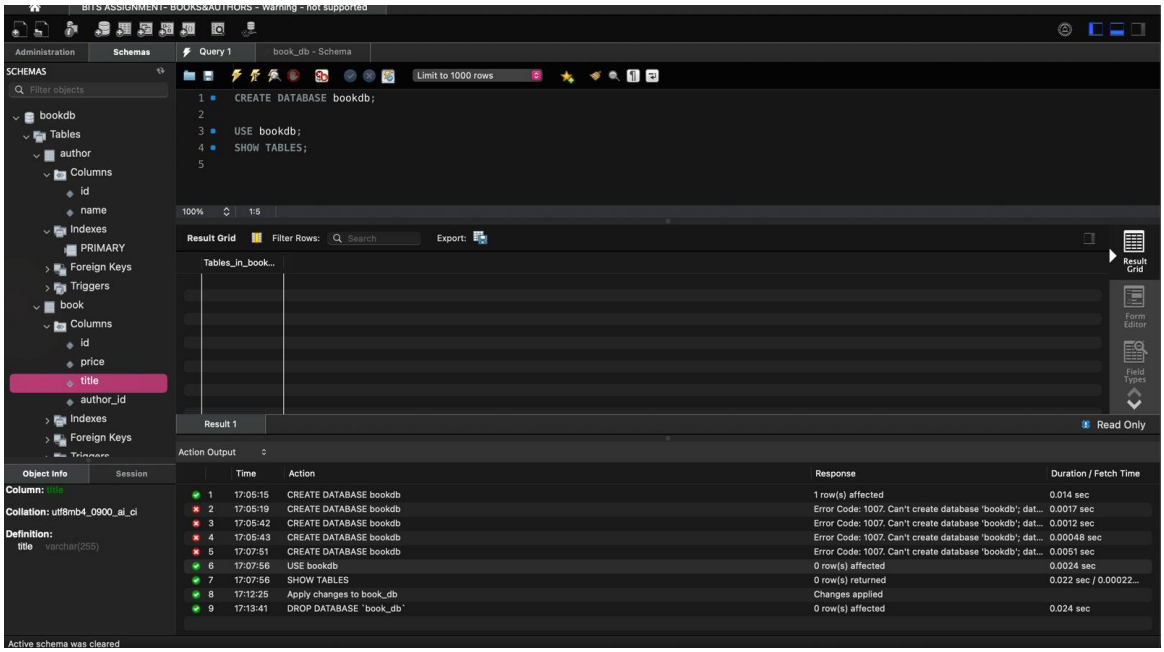
- ## CHALLENGES FACED

- JSP configuration with Spring Boot
- Handling port conflicts
- Fixing edit/update mapping issues
- Handling null pointer exceptions

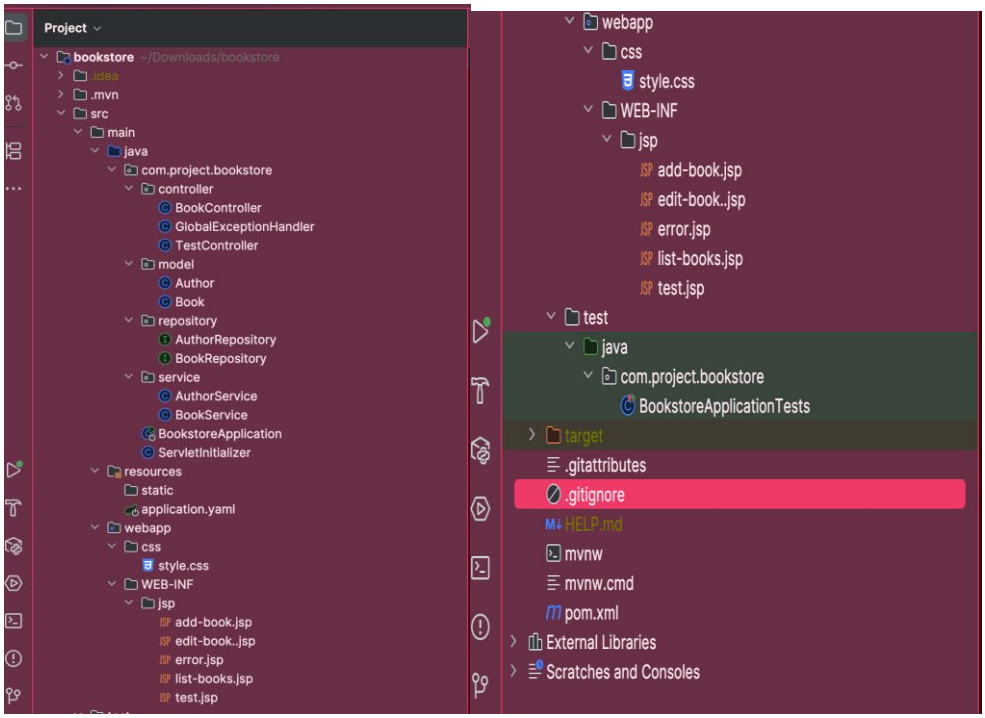
GITHUB LINK --->

https://github.com/Sanskriti10247/spring-boot-bookstore-crud.git

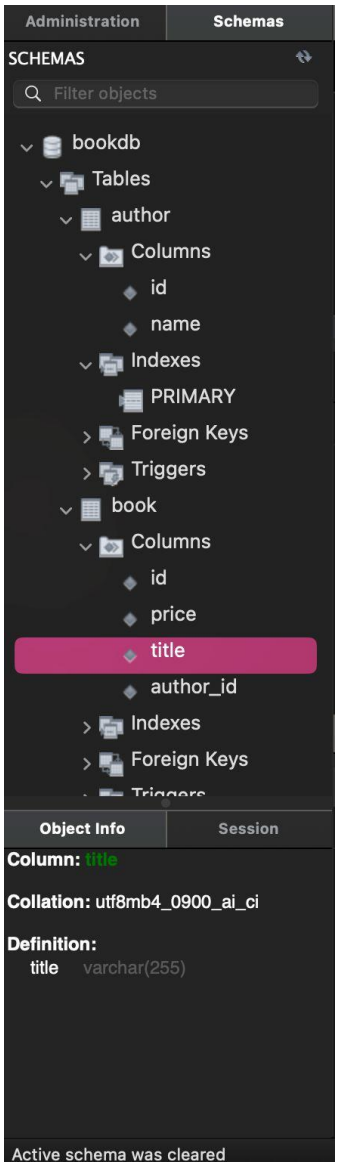
SCREENSHOTS FOR REFERENCE :-->



FOLDER STRUCTURE :

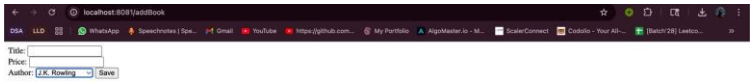


DB :



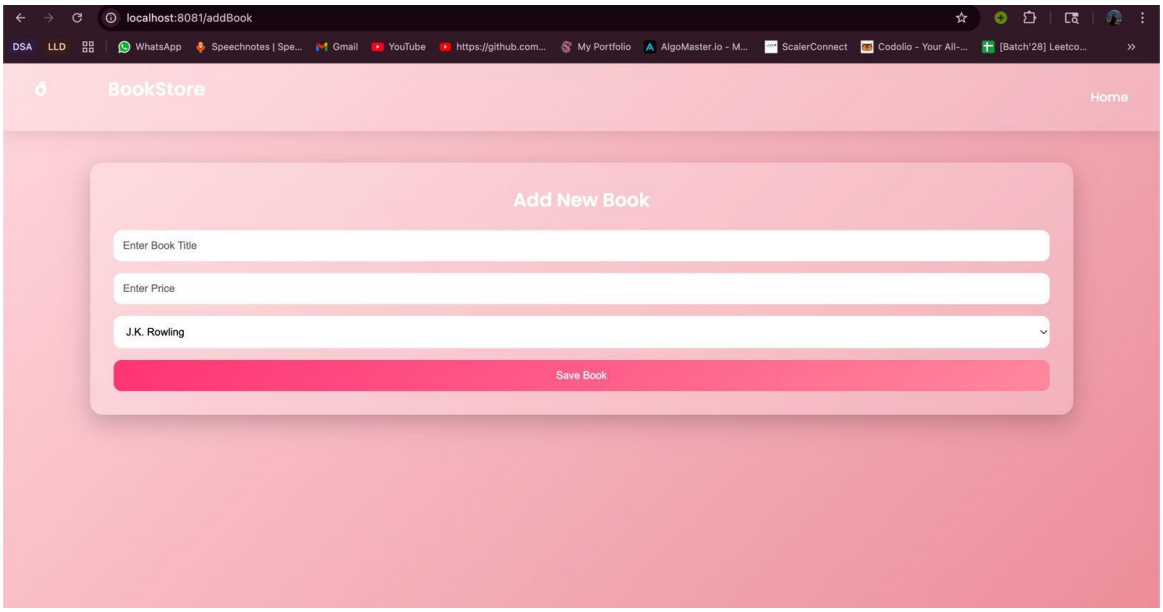
OUTPUTS->

Without UI (Initially)->

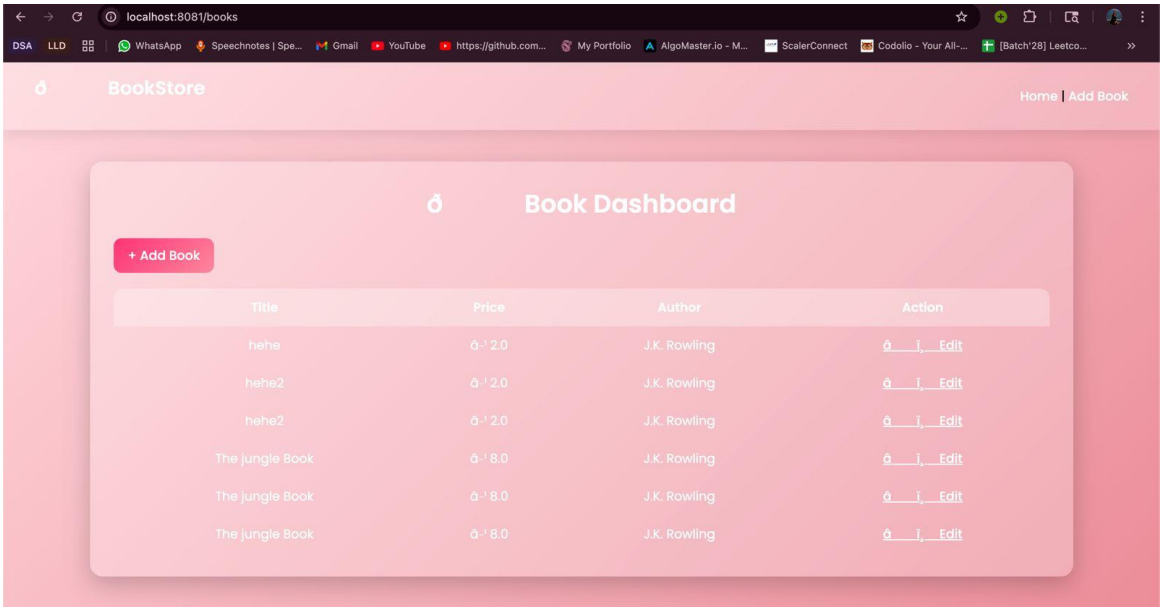


After UI Changes->

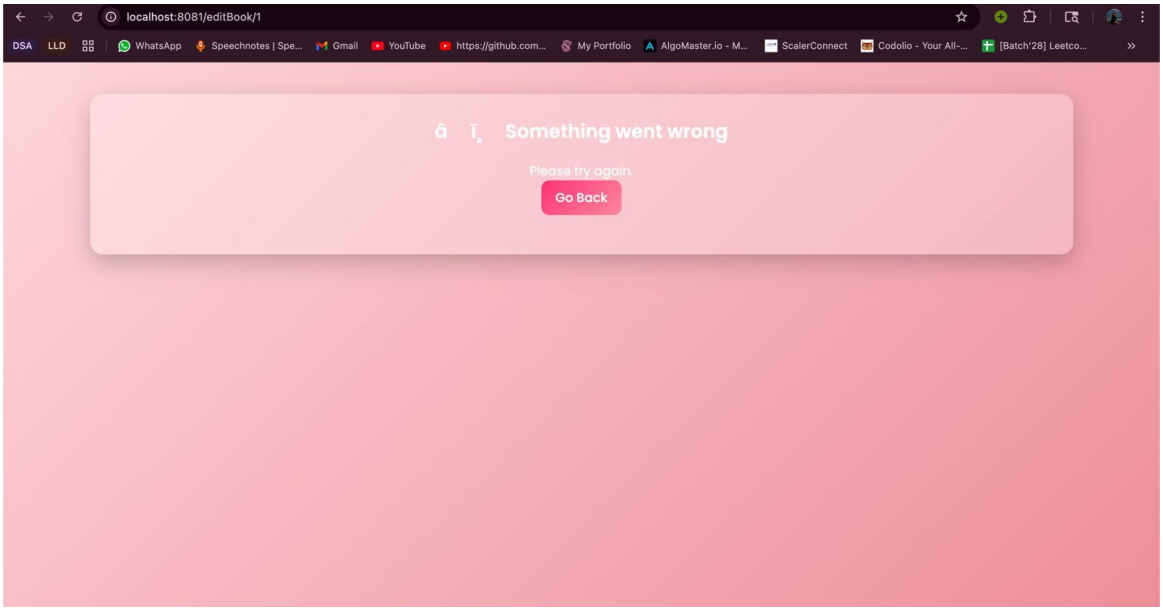
1. ADD BOOK :



2. DASHBAORD SHOWING ALL THE BOOK COLLECTIONS:



3. EDIT BOOKS :



TABLES AND THEIR RESULT IN MY SQL WORKBENCH:

SQL COMMANDS -->

1. SELECT FROM AUTHOR:

```
SELECT * FROM AUTHOR
```

2. SELECT FROM BOOKS:

```
SELECT * FROM BOOKS
```

3. (JOIN OPERATION) :

```
SELECT b.title, b.price, a.name
```

```
FROM book b
INNER JOIN author a ON b.author_id = a.id;
```

```
36  SELECT * FROM author;
100%  22:36
```

Result Grid Filter Rows: Search Edit: Export/Import:

id	name
1	J.K. Rowling
2	George Orwell
3	Agatha Christie
4	Chetan Bhagat
5	Dan Brown
6	Ruskin Bond
7	Arundhati Roy
8	Paulo Coelho
9	Jane Austen
10	Mark Twain
11	J.K. Rowling
12	George Orwell

author 6

Action Output

	Time	Action	Response
10	18:13:23	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Chetan Bhagat'), ('Paulo Coelho'), ('Dan Brown'), ('...	10 row(s) a
11	18:13:23	INSERT INTO book (title, price, author_id) VALUES ('Harry Potter', 500, 1), ('1984', 300, 2), ('Half Girlfriend', 250, 3), ('The A...	10 row(s) a
12	18:16:37	USE bookdb	0 row(s) aff
13	18:16:37	SHOW TABLES	2 row(s) ret
14	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Agatha Christie'), ('Chetan Bhagat'), ('Dan Brown'),...	10 row(s) a
15	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Chetan Bhagat'), ('Paulo Coelho'), ('Dan Brown'), ('...	10 row(s) a
16	18:16:37	INSERT INTO book (title, price, author_id) VALUES ('Harry Potter', 500, 1), ('1984', 300, 2), ('Half Girlfriend', 250, 3), ('The A...	10 row(s) a
17	18:16:37	SELECT * FROM author LIMIT 0, 1000	70 row(s) r
18	18:29:20	SELECT * FROM book LIMIT 0, 1000	36 row(s) r

```
38  SELECT * FROM book;
100%  20:38
```

Result Grid Filter Rows: Search Edit: Export/Import:

id	price	title	author_id
3	2	hehe2	1
4	8	The jungle Book	1
5	8	The jungle Book	1
6	8	The jungle Book	1
7	500	Harry Potter	1
8	300	1984	2
9	250	Half Girlfriend	3
10	350	The Alchemist	4
11	450	Da Vinci Code	5
12	400	Murder on Orie...	6
13	200	Malgudi Days	7
14	300	Pride and Prej...	8

book 7

Action Output

	Time	Action	Response
11	18:13:23	INSERT INTO book (title, price, author_id) VALUES ('Harry Potter', 500, 1), ('1984', 300, 2), ('Half Girlfriend', 250, 3), ('The A...	10 row(s) a
12	18:16:37	USE bookdb	0 row(s) aff
13	18:16:37	SHOW TABLES	2 row(s) ret
14	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Agatha Christie'), ('Chetan Bhagat'), ('Dan Brown'),...	10 row(s) a
15	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Chetan Bhagat'), ('Paulo Coelho'), ('Dan Brown'), ('...	10 row(s) a
16	18:16:37	INSERT INTO book (title, price, author_id) VALUES ('Harry Potter', 500, 1), ('1984', 300, 2), ('Half Girlfriend', 250, 3), ('The A...	10 row(s) a
17	18:16:37	SELECT * FROM author LIMIT 0, 1000	70 row(s) r
18	18:29:20	SELECT * FROM book LIMIT 0, 1000	36 row(s) r
19	18:29:52	SELECT * FROM author LIMIT 0, 1000	70 row(s) r

```
40 •  SELECT b.title, b.price, a.name
100%   43:42
```

Result Grid Filter Rows: Search Export:

title	price	name
hehe	2	J.K. Rowling
hehe2	2	J.K. Rowling
hehe2	2	J.K. Rowling
The jungle Book	8	J.K. Rowling
The jungle Book	8	J.K. Rowling
The jungle Book	8	J.K. Rowling
Harry Potter	500	J.K. Rowling
1984	300	George Orwell
Half Girlfriend	250	Agatha Christie
The Alchemist	350	Chetan Bhagat
Da Vinci Code	450	Dan Brown
Murder on Orie	400	Ruskin Bond

Result 8

Action Output

	Time	Action	Response
12	18:16:37	USE bookdb	0 row(s) aff
13	18:16:37	SHOW TABLES	2 row(s) ret
14	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Agatha Christie'), ('Chetan Bhagat'), ('Dan Brown'),...	10 row(s) a
15	18:16:37	INSERT INTO author (name) VALUES ('J.K. Rowling'), ('George Orwell'), ('Chetan Bhagat'), ('Paulo Coelho'), ('Dan Brown'), ('...	10 row(s) a
16	18:16:37	INSERT INTO book (title, price, author_id) VALUES ('Harry Potter', 500, 1), ('1984', 300, 2), ('Half Girlfriend', 250, 3), ('The A...	10 row(s) a
17	18:16:37	SELECT * FROM author LIMIT 0, 1000	70 row(s) r
18	18:29:20	SELECT * FROM book LIMIT 0, 1000	36 row(s) r
19	18:29:52	SELECT * FROM author LIMIT 0, 1000	70 row(s) r
20	18:30:25	SELECT * FROM book LIMIT 0, 1000	36 row(s) r

JUNIT TEST:--->>

```java
package com.project.bookstore;

import com.project.bookstore.model.Book;
import com.project.bookstore.service.BookService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
class BookServiceTest {

    @Autowired
    private BookService service;

    @Test
    void testGetBooks() {
        List<Book> books = service.getAllBooks();
        assertNotNull(books);
    }

    @Test
    void testSaveBook() {
        Book b = new Book();
        b.setTitle("Test Book");
        b.setPrice(100);

        Book saved = service.save(b);
        assertNotNull(saved.getId());
    }
}
```

Conclusion

Successfully built a full-stack Spring Boot application with MVC architecture.